

Backlog Maître Ultra Détaillé — Sahar

★ **CE FICHIER EST TA SEULE SOURCE DE VÉRITÉ — ignore tous les autres documents précédents**

🔍 Quiz de repositionnement (fais ça EN PREMIER, avant toute autre chose)

Réponds honnêtement à chaque question pour savoir exactement où reprendre :

- `docker run hello-world` fonctionne dans ton terminal WSL Ubuntu ?
 - Non → va à la **Tâche 0.1**
 - Oui → continue à la question 2
- As-tu un dossier `~/iot_project/backend/` qui existe, avec au moins un fichier `.py` dedans ?
 - Non → va à la **Tâche 1.1** (Phase 0 est terminée pour toi)
 - Oui → continue à la question 3
- Ton `docker-compose up` lance une API FastAPI qui répond sur `localhost:8000` ?
 - Non → va à la **Tâche 2.1**
 - Oui → continue à la question 4
- As-tu déjà une fonction qui détecte le framework (Zephyr/Arduino/ESP-IDF) d'un vrai dépôt GitHub ?
 - Non → va à la **Tâche 3.1**
 - Oui → va à la **Tâche 4.1**

D'après notre conversation, tu es très probablement entre la fin de la Phase 0 et le tout début de la Phase 1 — c'est-à-dire que l'installation est faite, mais aucun fichier du projet lui-même n'existe encore. Commence donc à la **Tâche 1.1**.

📅 Planning réaliste sur 4 semaines (mis à jour, coupures de courant prises en compte)

Semaine	Phases à couvrir	Si le temps presse encore plus, coupe...
1	Fin Phase 0 (si besoin) + Phase 1 (Docker de base) + Phase 2 (FastAPI)	Rien à couper ici, c'est le socle
2	Phase 3 complète (Agent 1 multi-frameworks)	Teste sur 2-3 dépôts réels au lieu de 4-5 (Tâche 3.8)

Semaine	Phases à couvrir	Si le temps presse encore plus, coupe...
3	Phase 4 (Mosquitto/InfluxDB/Grafana) + début Phase 5 (RAG)	Saute Mbed OS dans les FINGERPRINTS (Tâche 3.3) — Zephyr/Arduino/ESP-IDF suffisent
4	Fin Phase 5 (RAG) + Phase 6 (finitions) + intégration équipe	Le jeu de test RAG (Tâche 5.8) peut se limiter à 2 questions au lieu de 5

⚠ Ce planning suppose un rythme soutenu mais réaliste. Si une semaine déborde à cause d'une coupure de courant, ne rattrape pas en sacrifiant le sommeil — coupe plutôt les éléments listés dans la colonne de droite, ce sont les seuls sacrificiables sans casser la démo finale.

Comment utiliser ce document

1. Suis les tâches **dans l'ordre exact**, ne saute rien.
2. Chaque tâche a une case ☒ "C'est terminé quand..." — n'avance pas tant que ce n'est pas vrai.
3. Chaque tâche a une ligne "💬 **Demande à Cline :**" — le prompt exact à copier-coller, déjà calibré pour qu'il t'explique au lieu de tout faire à ta place.
4. Ce document remplace toutes les versions précédentes de ton backlog — ne les utilise plus.

PHASE 0 — Environnement (WSL2, Docker, Ollama, Cline)

Tâche 0.1 — Installer VS Code (Windows)

Télécharge sur <https://code.visualstudio.com/>, installe avec les réglages par défaut. ☒

Terminé quand : VS Code s'ouvre normalement.

Tâche 0.2 — Installer WSL2 (PowerShell, une seule fois)

Ouvre **Windows Terminal en administrateur**, onglet PowerShell :

```
powershell

wsl --install -d Ubuntu
```

Redémarre le PC si demandé. Rouvre Windows Terminal, un nouvel onglet "Ubuntu" apparaît — clique dessus, crée un nom d'utilisateur et un mot de passe **quand ils te sont demandés dans cette même fenêtre**.

✅ **Terminé quand :** tu as un prompt du style `sahar_jerbi@sahar:~$` (pas `PS C:\...\>`).

Tâche 0.3 — Vérifier WSL2 (PowerShell)

```
powershell  
  
wsl -l -v
```

✅ **Terminé quand :** ça affiche `Ubuntu`, `Running`, `VERSION 2`.

Tâche 0.4 — Installer Docker Desktop (Windows) + activer l'intégration WSL

1. Télécharge sur <https://www.docker.com/products/docker-desktop/>, installe.
2. Ouvre Docker Desktop → icône engrenage (Settings) → **Resources** → **WSL Integration** → active le toggle à côté de "Ubuntu" → Apply & Restart.

✅ **Terminé quand (dans le terminal Ubuntu) :**

```
bash  
  
docker --version  
docker run hello-world
```

affiche un message de bienvenue.

Tâche 0.5 — Installer Python (terminal Ubuntu)

```
bash  
  
sudo apt update  
sudo apt install -y python3 python3-pip python3-venv git curl  
python3 --version
```

✅ **Terminé quand :** un numéro de version Python s'affiche.

Tâche 0.6 — Créer ton dépôt GitHub et le cloner dans WSL

1. Sur <https://github.com>, crée un nouveau dépôt (ex: `iot-project`).
2. Dans le terminal Ubuntu :

```
bash  
  
cd ~  
git clone https://github.com/ton-compte/iot-project.git  
cd iot-project
```

✅ **Terminé quand :** `pwd` affiche `/home/sahar_jerbi/iot-project`.

Tâche 0.7 — Ouvrir le projet dans VS Code, connecté à WSL

Dans le terminal Ubuntu, depuis le dossier du projet :

```
bash
code .
```

✅ **Terminé quand :** VS Code s'ouvre, et le coin en bas à gauche affiche "WSL: Ubuntu".

Tâche 0.8 — Installer les extensions VS Code (dans WSL, pas Windows)

Panneau Extensions (`Ctrl+Shift+X`). Pour chacune ci-dessous, si un bouton "Install in WSL: Ubuntu" apparaît, clique CELUI-LÀ (pas le bouton Install normal) :

- Python (Microsoft)
- Docker (Microsoft)
- GitLens
- Error Lens
- YAML
- Thunder Client

■ **Pourquoi ce bouton spécifique existe :** ton code vit dans WSL, donc les extensions doivent aussi "vivre" là où est ton code pour pouvoir l'analyser. Une extension installée côté Windows ne voit pas les fichiers dans WSL.

✅ **Terminé quand :** aucune extension n'affiche "disabled, install in WSL" dans son onglet.

Tâche 0.9 — Installer Ollama + ton modèle local (terminal Ubuntu)

```
bash
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5-coder:7b
ollama pull nomic-embed-text
```

⌚ Le premier téléchargement prend du temps (plusieurs Go) — laisse tourner, ne lance pas d'autre commande Ollama en parallèle (ça peut interrompre le téléchargement).

✅ **Terminé quand :**

```
bash
```

```
ollama list
```

affiche les deux modèles.

Tâche 0.10 — Limiter la RAM que WSL2 peut utiliser (PowerShell)

```
powershell  
  
notepad "$env:USERPROFILE\.wslconfig"
```

Colle (ajuste selon ta RAM totale) :

```
[ws12]  
memory=12GB  
processors=4
```

Enregistre, puis :

```
powershell  
  
wsl --shutdown
```

Rouvre ton terminal Ubuntu.

Tâche 0.11 — Installer et connecter Cline

1. Extensions VS Code → cherche "Cline" → Install in WSL.
2. Clique l'icône Cline dans la barre latérale → réglages (engrenage).
3. Provider : **Ollama**, URL : `http://localhost:11434`, modèle : `qwen2.5-coder:7b`.
4. **Si trop lent sur ta machine** (normal avec un GPU intégré Intel) : bascule le provider sur DeepSeek (gratuit, plus rapide) dans les mêmes réglages.

✅ **Terminé quand** : tu tapes "crée un fichier hello.py qui affiche Bonjour" dans Cline, et il propose le fichier (tu n'as pas encore besoin d'approuver, juste vérifier qu'il répond).

Tâche 0.12 — Comment donner du contexte à Cline (fais-le maintenant, une fois pour toutes)

Copie-colle ce prompt dans Cline pour qu'il comprenne ton projet et respecte ta façon d'apprendre :

```
[Utilise le prompt de contexte complet qu'on a déjà préparé ensemble –  
copie-le depuis notre conversation précédente, il décrit le projet,  
ton périmètre (Docker + Agent 1 + Agent 4), et la règle : m'expliquer  
étape par étape plutôt que tout faire à ma place.]
```

✅ **Terminé quand :** Cline répond en confirmant qu'il a compris ton rôle et la règle "expliquer avant de coder".

Tâche 0.13 — Copier tes documents de référence dans le projet

```
bash

mkdir -p ~/iot-project/reference-docs
```

Copie ton cahier des charges, ton backlog, tes guides (les fichiers `.md` qu'on a créés ensemble) dedans, soit via `cp /mnt/c/Users/.../fichier.md ~/iot-project/reference-docs/`, soit en les glissant dans VS Code (voir solutions du début de ce message).

✅ **Terminé quand :** ces fichiers apparaissent dans la sidebar VS Code, sous `reference-docs/`.

PHASE 1 — Docker : les bases

Tâche 1.1 — Créer la structure de dossiers

```
bash

cd ~/iot-project
mkdir -p backend/app
```

✅ **Terminé quand :** `backend/app/` existe (visible dans la sidebar VS Code).

Tâche 1.2 — Premier script Python

Crée `backend/app/main.py` (clic droit sur `app` → New File) :

```
python

print("Ma plateforme démarre")
```

Teste :

```
bash

python3 backend/app/main.py
```

✅ **Terminé quand :** le message s'affiche dans le terminal.

Tâche 1.3 — Premier Dockerfile

Crée `backend/Dockerfile` :

dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY app/main.py .
CMD ["python", "main.py"]
```

💬 **Demande à Cline :** *"Explique-moi ligne par ligne ce que fait ce Dockerfile, sans le modifier."*

Tâche 1.4 — Construire et lancer l'image

```
bash

docker build -t mon-backend ./backend
docker run mon-backend
```

✅ **Terminé quand :** le message s'affiche, exécuté depuis un conteneur.

Tâche 1.5 — Premier docker-compose.yml

Crée `docker-compose.yml` à la racine :

```
yaml

services:
  backend:
    build: ./backend
```

```
bash

docker compose up
```

✅ **Terminé quand :** ça fonctionne pareil, via Compose.

Tâche 1.6 — Sécuriser tes futurs secrets dès maintenant

Crée `.gitignore` à la racine :

```
.env
venv/
__pycache__/
rag_db/
```

Crée `.env` (vide pour l'instant) :

✓ Terminé quand :

```
bash

git status
```

ne montre jamais `(.env)` dans les fichiers à commiter.

PHASE 2 — FastAPI

Tâche 2.1 — Environnement virtuel Python

```
bash

cd backend
python3 -m venv venv
source venv/bin/activate
```

✓ Terminé quand : le prompt affiche `(venv)` au début.

Tâche 2.2 — Installer les dépendances

```
bash

pip install fastapi "uvicorn[standard]" python-dotenv anthropic gitpython
pip freeze > requirements.txt
```

Tâche 2.3 — API "Hello"

Remplace `(backend/app/main.py)` :

```
python

from fastapi import FastAPI

app = FastAPI(title="IoT Backend - Sahar")

@app.get("/")
def racine():
    return {"message": "Backend is running"}
```

```
bash
```



```
uvicorn app.main:app --reload --host 0.0.0.0
```

Ouvre `http://localhost:8000` dans ton navigateur (Windows, ça marche automatiquement depuis WSL).

✅ **Terminé quand :** tu vois `{"message": "Backend is running"}`.

Tâche 2.4 — Route POST /analyze (vide, testée avec Thunder Client)

```
python

from pydantic import BaseModel

class Requete(BaseModel):
    url_github: str

@app.post("/analyze")
def analyser(requete: Requete):
    return {"statut": "Projet reçu", "url": requete.url_github}
```

Dans Thunder Client (icône dans la sidebar) : nouvelle requête POST vers `http://localhost:8000/analyze`, corps JSON `{"url_github": "test"}`.

✅ **Terminé quand :** Thunder Client affiche la réponse attendue.

Tâche 2.5 — Dockeriser FastAPI

```
dockerfile

# backend/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
yaml
```

```
# docker-compose.yml
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    env_file:
      - .env
```

```
bash

cd ..
docker compose up --build
```

✅ **Terminé quand :** `http://localhost:8000/docs` fonctionne depuis le conteneur.

💡 **Demande à Cline (si bloquée) :** "Voici l'erreur exacte : [colle l'erreur]. Explique-moi ce qu'elle signifie avant de proposer une correction."

PHASE 3 — Agent 1 : détection multi-frameworks

Tâche 3.1 — Cloner un dépôt GitHub par script

Crée `backend/app/agent1.py` :

```
python

from git import Repo, GitCommandError
import tempfile, shutil

def cloner_depot(url_github: str) -> dict:
    dossier_temp = tempfile.mkdtemp()
    try:
        Repo.clone_from(url_github, dossier_temp, depth=1)
        return {"succes": True, "dossier": dossier_temp}
    except GitCommandError as e:
        shutil.rmtree(dossier_temp, ignore_errors=True)
        return {"succes": False, "erreur": str(e)}
```

✅ **Terminé quand :** testé avec une vraie URL, `succes: True` et le dossier contient des fichiers.

Tâche 3.2 — Lister tous les fichiers du dépôt (en ignorant les dépendances)

```
python
```

```
import os

def lister_tous_les_fichiers(dossier: str) -> set:
    fichiers_trouves = set()
    for racine, dossiers, fichiers in os.walk(dossier):
        if "deps" in racine or "modules" in racine or ".git" in racine:
            continue
        fichiers_trouves.update(fichiers)
    return fichiers_trouves
```

Tâche 3.3 — Définir les empreintes de chaque framework

python

```
FINGERPRINTS = {
    "Zephyr RTOS": {"fichiers_requis": ["prj.conf"], "fichiers_bonus": ["CMakeL
    "Arduino/PlatformIO": {"fichiers_requis": ["platformio.ini"], "fichiers_bon
    "ESP-IDF": {"fichiers_requis": ["sdkconfig"], "fichiers_bonus": ["CMakeList
    "Mbed OS": {"fichiers_requis": ["mbed_app.json"], "fichiers_bonus": []},
}
```

💡 **Demande à Cline:** *"Explique-moi pourquoi CMakeLists.txt seul ne suffit pas à distinguer Zephyr d'ESP-IDF, sans modifier mon code."*

Tâche 3.4 — Scorer et détecter le framework

python

```
def detecter_framework(fichiers_trouves: set) -> dict:
    resultats = {}
    for framework, regles in FINGERPRINTS.items():
        requis_presents = [f for f in regles["fichiers_requis"] if f in fichiers_trouves]
        bonus_presents = [f for f in regles["fichiers_bonus"] if f in fichiers_trouves]
        if requis_presents:
            resultats[framework] = {
                "score": len(requis_presents) * 2 + len(bonus_presents),
                "fichiers_detectes": requis_presents + bonus_presents
            }
    if not resultats:
        return {"framework": "inconnu", "confiance": "basse"}
    meilleur = max(resultats, key=lambda f: resultats[f]["score"])
    return {
        "framework": meilleur, "confiance": "haute",
        "fichiers_detectes": resultats[meilleur]["fichiers_detectes"]
    }
```

✓ Terminé quand :

python

```
print(detecter_framework({"platformio.ini", "src", "README.md"}))
# → {"framework": "Arduino/PlatformIO", "confiance": "haute", ...}
```

Tâche 3.5 — Extraire les protocoles (si Zephyr)

python

```
def valider_prj_conf(chemin: str) -> dict:
    with open(chemin, "r", errors="ignore") as f:
        contenu = f.read()
    if len(contenu.strip()) == 0:
        return {"valide": False, "raison": "fichier vide"}
    protocoles = []
    if "CONFIG_MQTT" in contenu:
        protocoles.append("MQTT")
    if "CONFIG_WIFI" in contenu or "CONFIG_NET_L2_WIFI" in contenu:
        protocoles.append("WiFi")
    return {"valide": True, "protocoles_detectes": protocoles}
```

Tâche 3.6 — LLM seulement si ambigu, fonction complète

python

```
def analyser_avec_ia_si_besoin(fichiers_trouves: set, resultat_simple: dict) ->
    if resultat_simple["confiance"] == "haute":
        return resultat_simple
    # Appel LLM local via Ollama (gratuit) si détection simple échoue
    import ollama, json
    prompt = f"""
    Fichiers trouvés : {list(fichiers_trouves)}
    Aucune règle simple n'a identifié le framework.
    Réponds en JSON : {{"framework": "...", "confiance": "moyenne"}}
    """
    reponse = ollama.chat(model="qwen2.5-coder:7b", messages=[{"role": "user",
    return json.loads(reponse["message"]["content"])]
```

Tâche 3.7 — Fonction complète bout en bout

python

```
def analyser_depot_complet(url_github: str) -> dict:
    clonage = cloner_depot(url_github)
    if not clonage["succes"]:
        return {"erreur": clonage["erreur"]}
    fichiers_trouves = lister_tous_les_fichiers(clonage["dossier"])
    resultat = detecter_framework(fichiers_trouves)
    return analyser_avec_ia_si_besoin(fichiers_trouves, resultat)
```

Tâche 3.8 — Tester sur 4-5 vrais dépôts GitHub différents

Cherche sur GitHub : un exemple Zephyr, un exemple PlatformIO, un exemple ESP-IDF.
Note dans un tableau ce qui marche.

✅ **Terminé quand :** au moins 3 dépôts réels sont correctement identifiés.

Tâche 3.9 — Brancher à FastAPI et publier le contrat

python

```
from app.agent1 import analyser_depot_complet

@app.post("/analyze")
def analyser(requete: Requete):
    return analyser_depot_complet(requete.url_github)
```

Crée `contracts/agent1_output.json` avec un exemple réel, commit, push, préviens Ameni.

PHASE 4 — Étendre l'infrastructure Docker

Tâche 4.1 — Ajouter Mosquitto

```
yaml

mosquitto:
  image: eclipse-mosquitto:2
  ports:
    - "1883:1883"
  volumes:
    - ./mosquitto/config:/mosquitto/config
```

Crée `mosquitto/config/mosquitto.conf`:

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
```

```
bash

docker compose up -d
```

Tâche 4.2 — Ajouter InfluxDB

```
yaml

influxdb:
  image: influxdb:2.7
  ports:
    - "8086:8086"
  environment:
    - DOCKER_INFLUXDB_INIT_MODE=setup
    - DOCKER_INFLUXDB_INIT_USERNAME=admin
    - DOCKER_INFLUXDB_INIT_PASSWORD=motdepasse123
    - DOCKER_INFLUXDB_INIT_ORG=iot-org
    - DOCKER_INFLUXDB_INIT_BUCKET=capteurs
  volumes:
    - influx_data:/var/lib/influxdb2

volumes:
  influx_data:
```

Tâche 4.3 — Ajouter Grafana

yaml

```
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
```

✅ Terminé quand :

bash

```
docker compose up -d
docker ps
```

montre 4 conteneurs actifs.

PHASE 5 — Agent 4 : RAG avec embeddings locaux (Ollama, gratuit)

Tâche 5.1 — Installer les bibliothèques

bash

```
pip install chromadb langchain-text-splitters ollama
pip freeze > requirements.txt
```

Tâche 5.2 — Créer un document de test

Crée `docs/erreur_test.md` :

markdown

```
# Erreur : west command not found
Solution : `pip install west` puis redémarrer le terminal.
```

Tâche 5.3 — Générer des embeddings avec Ollama (gratuit, local)

python

```
import ollama
import chromadb

client_chroma = chromadb.PersistentClient(path="./rag_db")
collection = client_chroma.get_or_create_collection("docs_techniques")

def generer_embedding(texte: str) -> list:
    reponse = ollama.embeddings(model="nomic-embed-text", prompt=texte)
    return reponse["embedding"]
```

Tâche 5.4 — Indexer un document (chunking + embeddings)

```
python

from langchain_text_splitters import RecursiveCharacterTextSplitter

def indexer_document(chemin_fichier: str, source: str):
    with open(chemin_fichier, "r") as f:
        texte = f.read()
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_text(texte)
    embeddings = [generer_embedding(c) for c in chunks]
    collection.add(documents=chunks, embeddings=embeddings, ids=[f"{source}_{i}" for i in range(len(chunks))])

indexer_document("docs/erreur_test.md", "test")
```

✅ **Terminé quand :** aucune erreur, message de confirmation.

Tâche 5.5 — Rechercher et générer une réponse

```
python
```



```
def demander_a_assistant(question: str) -> str:
    embedding_q = generer_embedding(question)
    resultats = collection.query(query_embeddings=[embedding_q], n_results=3)
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])
    prompt = f"""
    Extraits : {chunks_pertinents}
    Question : {question}
    Réponds uniquement à partir de ces extraits. Si absent, dis-le clairement.
    """
    reponse = ollama.chat(model="qwen2.5-coder:7b", messages=[{"role": "user",
    return reponse["message"]["content"]

print(demander_a_assistant("J'ai une erreur west command not found, que faire ?
```

✓ Terminé quand : la réponse mentionne `pip install west`.

Tâche 5.6 — Ajouter le raisonnement multi-étapes (diagnostic industriel)

```
python

from datetime import datetime
import json

def raisonnement_diagnostic(mesure: dict, seuils: dict) -> dict:
    if mesure["temperature"] <= seuils["temperature_max"]:
        return {"statut": "normal", "diagnostic": None}

    embedding_q = generer_embedding(f"limite de température pour {mesure['capteur']}")
    resultats = collection.query(query_embeddings=[embedding_q], n_results=3)
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])

    prompt = f"""
    Mesure : {mesure}, seuil : {seuils}
    Documentation : {chunks_pertinents}
    Explique l'anomalie et propose une recommandation. Réponds en JSON avec
    les champs "diagnostic" et "recommandation".
    """
    reponse = ollama.chat(model="qwen2.5-coder:7b", messages=[{"role": "user",
    resultat = json.loads(reponse["message"]["content"])
    resultat["horodatage"] = str(datetime.now())
    return resultat
```

Tâche 5.7 — Ajouter 4-5 vrais documents (erreurs Docker, MQTT, Zephyr)

Répète 5.2-5.4 avec du vrai contenu rencontré par l'équipe.

Tâche 5.8 — Construire un jeu de test (évaluation)

```
python

jeu_de_test = [
    {"question": "erreur west command not found", "attendu": "pip install west"}
]

for cas in jeu_de_test:
    reponse = demander_a_assistant(cas["question"])
    print("✅" if cas["attendu"] in reponse else "❌", cas["question"])
```

Tâche 5.9 — Brancher à FastAPI

```
python

class QuestionRAG(BaseModel):
    question: str

@app.post("/assistant")
def assistant(requete: QuestionRAG):
    return {"reponse": demander_a_assistant(requete.question)}
```

PHASE 6 — Finitions

Tâche 6.1 — Nettoyer et documenter

Vérifie que `docker compose up -d` démarre tout sans erreur. Complète le README.

Tâche 6.2 — Relire la Pull Request CI/CD d'Ameni

Comprends chaque étape, pose des questions.

Tâche 6.3 — Commit final et présentation à l'équipe

```
bash

git add .
git commit -m "Docker infra + Agent 1 multi-framework + Agent 4 RAG"
git push origin main
```

Une seule règle à garder en tête à chaque tâche : simple d'abord (règles, fichiers), IA seulement si nécessaire, test sur du réel à la fin.